

Web Based Java

Day 1:

JDBC:

JDBC stands for Java to DB Connectivity.

It is an API used to bridge the gap between a Java Application and a Database.

JDBC allows applications to interact with DB for handling various operations commonly known as CRUD operations.

JDBC Architecture:

The JDBC architecture mainly consists of an entity known as a **Driver**.

Every DB Vendor (Product Provider) provides its own API that simplifies access to all the client-side applications which want to interact with DB. It is known as Vendor Specific API.

A Driver is a program that converts JDBC calls into some format that is compatible with the DB Vendor Specific API.

Typically, in JDBC there are 4 types of drivers:

1. Type 1
2. Type 2
3. Type 3
4. Type 4

Type 1:

It is called as JDBC-ODBC Bridge.

It uses a 3rd party library ODBC (Open DB Connectivity) given by Microsoft.

It converts JDBC calls into ODBC compatible format and then further conversion takes place through ODBC driver.

Limitations:

1. It is the slowest
2. It is platform dependent
3. Every client machine needs ODBC configuration setup

It is good for small and simple applications but not recommended for large scale applications.

Type 2:

It is called as Native API, Partly Java Driver.

Unlike Type 1, it does not use any 3rd party library and hence it is faster as compared to Type 1. It is platform independent.

Its implementation is a combination of Java and DB vendor specific standards.

It uses DB specific native API to make the calls and hence on every client machine, the DB specific native API must be installed.

Type 3:

It is called as Net Protocol, Intermediate DB Access Server.

It allows applications to interact with multiple DB servers.

It uses some middleware that is known as Intermediate DB Access Server which uses its own driver to interact with multiple databases.

Type 4:

It is called as Vendor Specific, Pure Java Driver.

It is completely implemented using Java standards but provided by a DB Vendor.

It uses TCP Socket connections to interact with DB.

Benefits:

1. It is the fastest
2. It is platform independent
3. It does not require any setup on client machine.

JDBC Core API:

The core API of JDBC belongs to `java.sql` package.

It provides following:

1. *DriverManager* (Class)
2. *Connection*
3. *Statement*
4. *PreparedStatement*
5. *CallableStatement*
6. *ResultSet*

JDBC Steps:

1. Load the driver
2. Establish Connection
3. Obtain some Statement
4. Execute SQL Query
5. In case of SELECT query, obtain a *ResultSet* and perform navigation

Why JDBC Driver needs to be loaded?

In order to establish *Connection*, `java.sql` package provides a class: *DriverManager*.

It provides a method *getConnection()* that builds a connection and returns its reference.

However, it can establish the connection only when the relevant JDBC driver is registered with the *DriverManager* itself.

Generally, every JDBC driver specific implementation class provides a *static* block that handles the logic for registration.

Therefore, to perform registration, it is necessary to invoke this *static* block. It is possible using *Class.forName(..)* statement.

Obtaining a Statement:

Once a Connection is established, it can be used to perform query operations.

To perform query operations, some *Statement* is to be obtained.

The type of *Statement* depends upon the type of SQL query.

In general, in the context of JDBC, queries are of 2 types:

1. Simple Query (Query without Parameter)
2. Parameterized Query

In case of simple queries, *Statement* is used.

ResultSet:

It is an object present on Java side that maintains a data retrieved from DB table.

Just like table, it maintains data in the form of rows and columns.

Every row has a record position and every column has a column index where both of them start with 1 and not 0.

The column index depends upon the query and not the original DB table design.

In case of record positions, apart from the actual positions, there are 2 additional positions available:

1. Before First
2. After Last

By default, the cursor of the *ResultSet* points to Before First position.

In order to move the cursor in the forward direction, *next()* method is used.

It checks whether the next record is available or not and returns a boolean value accordingly.

If the record is available, shifts the cursor to that record.

In order to read column values, *ResultSet* interface provides several getter methods e.g. *getString()*, *getInt()*, *getDouble()* etc.

JDBC Standard Practices:

1. Code Separation for establishing a connection
2. Auto-Loading of JDBC Driver
3. Auto-Closing of resources like Connection, Statement, ResultSet etc.
4. Using a DAO Pattern (Data Access Object)

DAO Pattern:

It is a commonly used design pattern for handling DB connectivity.

Typically, it starts with a generic interface and the implementation classes specify the actual type.

Working with Parameterized Queries:

A query may have one or many parameters.

It is possible to execute parameterized queries using *Statement*; but in this case, as soon as the parameter value changes, a new version of query is formed and the query gets compiled every time.

This leads to performance problem. In order to overcome this problem, *PreparedStatement* is used.

It does not create the query repeatedly; rather it caches the query and therefore the query is compiled just once which is known as pre-compiled query.

In case of parameterized query, the unknown parameters are denoted using '?' and every parameter has an index that starts with 1, increments by 1 from left to right.

Before executing the query, it is necessary to substitute actual values in place of '?'.

PreparedStatement interface provides several setter methods e.g. *setInt()*, *setString()*, *setDouble()* and so on.

Day 2:

Performing DML Operations:

In case of DML operations like INSERT, UPDATE or DELETE, there is no need of *ResultSet* as these operations do not fetch any data; rather they modify the state of data.

In order to make a JDBC call for DML operations, *executeUpdate()* method is used.

It returns *int* that indicates the number of records affected due to the query that is fired.

Performing DML UPDATE:

In case of DML UPDATE, first it is necessary to load an entity on which UPDATE is required. This is done against ID.

Once an entity is loaded, its state can be modified using relevant setter methods.

However, this changes the state of an entity on Java side but not on DB side.

In order to change the state on DB side, the modified entity needs to be sent to DAO implementation for further updates.

Making a JDBC Call:

In order to make a JDBC call, *Statement* interface provides several methods e.g. *executeQuery()*, *executeUpdate()* and so on.

The *executeQuery()* method is generally used in case of SELECT queries. It returns a reference of type *ResultSet*.

The *executeUpdate()* method is generally used in case of DML operations. It returns *int* that indicates the number of records affected due to the query.

Apart from these, there are several other methods available to make a JDBC call e.g. *execute()*.

The *execute()* method is used for handling several requirements like executing DDL queries, executing dynamic queries etc.

It returns *boolean* which indicates whether the outcome of the query is a *ResultSet* or not.

Types of Statements:

In JDBC, statements are of 3 types:

1. *Statement*
2. *PreparedStatement*
3. *CallableStatement*

Statement is used to execute simple queries.

PreparedStatement is used to execute parameterized queries.

CallableStatement is used to execute stored procedures present on the backend DB side.

Transaction Management:

Transaction is a set of operations that must execute in a single unit.

Transactions must be atomic (Execute ALL or NONE) otherwise the data is left in inconsistent state.

When any DML operation is performed using JDBC, by default, changes are auto-committed.

Therefore, to manage transactions effectively, it is necessary to disable auto-commit. *Connection* interface provides *setAutocommit(boolean)* method to achieve this.

E.g.

```
dbConn.setAutocommit(false);
```

Once auto-commit is disabled, at any time changes can be committed or rolled back using *commit()* and *rollback()* methods.

Jakarta EE:

Jakarta EE i.e. Jakarta Enterprise Edition is a platform meant for developing web based as well as enterprise applications.

Web Application:

Any application that is accessible using a web browser is referred as a web application. It can be either internet based or intranet based.

Enterprise Application:

Any application that needs enterprise services to be integrated then that application is referred as an enterprise application.

These enterprise services are the higher end services e.g.

1. Scalability
2. Availability
3. Transaction Management
4. Persistency
5. Concurrency
6. Security
7. Asynchronous Messaging

Jakarta EE is completely based upon Component Driven Architecture.

Component:

A component is an application level reusable piece of code.

A component is of 2 types: Unmanaged and Managed.

Unmanaged Component:

When a component is to be instantiated explicitly using **new** keyword then that component is called as an Unmanaged Component.

Generally, Java SE consists of Unmanaged Components.

Managed Component:

When a component gets instantiated implicitly without **new** keyword then that component is called as Managed Component.

Generally, Jakarta EE consists of Managed Components.

Managed Components are divided into 2 types:

1. Web Component
2. Business Component

Web Component:

When a component is responsible for accepting a web request and generating a web response then that component is called as a Web Component.

A web component is of 2 types: Servlet and JSP

Business Component:

When a component is responsible for handling business logic of the application then that component is called as a Business Component.

It is taken care by EJB i.e. Enterprise Java Bean

How Jakarta EE components are managed?

Jakarta EE components are managed by some runtime environment known as Container.

Since there are 2 types of managed components, there are 2 types of containers:

1. Web Container
2. EJB Container

Web Container is responsible for managing web components like Servlet and JSP whereas EJB Container is responsible for managing business component like EJB.

How Containers are made available?

Containers are made available by 3rd party software known as servers.

Since there are 2 types of containers, there are 2 types of servers:

1. Web Server
2. Application Server

Web Server:

It is 3rd party software that provides web container.

There are several web servers available but the most popular one is TomCat given by Apache Software Foundation.

Application Server:

It is an extension to web server.

In addition to web container, it provides EJB container also.

There are several application servers available:

1. Weblogic (Oracle Corp)

2. Websphere (IBM)
3. Glassfish (Oracle Corp)
4. JBoss (RedHat)

Servlet:

It is a web component used to extend the functionality of a web server.

It runs on server side and hence can be used to handle server side processing.

It can be used to generate dynamic web contents.

Why Servlet?

There are several other technologies similar to servlet available however, servlets got wide acceptance as they are written in Java and hence can inherit all the features of Java language e.g. Platform Independent, Robust, Portable, Secured and so on.

Implementing Servlets:

In order to implement servlet, it is necessary to create a Java class that is referred as a Servlet Implementation Class.

This class has to depend upon some API which is commonly known as Servlet API.

The servlet API mainly consists of 2 packages:

1. *jakarta.servlet*
2. *jakarta.servlet.http*

There are 3 options available to define a servlet implementation class:

1. Implementing *Servlet* interface
2. Extending *GenericServlet* class
3. Extending *HttpServlet* class

If 1st option is used, it enforces to override all the methods of the interface and hence it is not much recommended.

The 2nd option is used when a servlet is required to handle any type of request (Protocol Independent).

The 3rd option is used when a servlet is required to handle only HTTP request (HTTP specific).

HTTP Basics:

HTTP stands for Hyper Text Transfer Protocol.

It is the most commonly used protocol in web applications.

It is a stateless protocol i.e. it cannot maintain any conversational state with the client.

Jakarta EE and Servers (Web Server and App Server):

Jakarta EE is a specification and not an implementation.

Its implementation is provided by 3rd party software like web server and application server.

Deployment:

When the web application has been developed, it needs to be deployed (hosted) on web server so that the server starts providing support for that application.

Life Cycle of Servlet:

The life cycle of servlet consists of 3 stages:

1. Instantiation and Initialization
2. Service
3. Destroy

Instantiation and Initialization:

When the servlet is requested very 1st time, it gets loaded by web container and once the loading is finished, it is instantiated by web container.

As soon as the servlet is instantiated, web container invokes *init()* method on that.

This method is meant for handling initialization if any.

After *init()*, web container invokes *service()* method.

This method is meant for serving the client.

Service:

When the servlet is requested, for every request, web container invokes a *service()* method.

Destroy:

When the server is stopped or the application is undeployed, web container removes the servlet instance and at that time, it invokes *destroy()* method.

Understanding *service()*:

It is the most significant method in servlet's life cycle because it gets called every time when the servlet is requested.

If the servlet specific class inherits *GenericServlet*, the *service()* method must be overridden because it is not implemented by *GenericServlet*.

If the servlet specific class inherits *HttpServlet*, the *service()* method need not to be overridden because *HttpServlet* already implements it.

The *service()* method implemented by *HttpServlet* class checks the type of incoming HTTP request.

HTTP Request can be of several types:

1. GET
2. POST
3. PUT

4. DELETE
5. OPTIONS
6. TRACE
7. HEAD
8. PATCH

Depending upon the type, it delegates the call to the respective doXxx() methods.

E.g. in case of GET, it invokes doGet(), in case of POST, it invokes doPost() and so on.

In order to customize the implementation, these methods need to be overridden in the subclass.

Requesting Servlet:

A servlet can be requested using several options:

1. Entering URL in browser's address bar (GET)
2. Using HTML Anchor Tag (<a href = "...") (GET)
3. Using HTML Form
4. From another servlet

Day 3:**HTML Form Processing:**

Sometimes, a client wants to send information towards server for further operations.

This is typically done using HTML form e.g. User Validation, User Registration.

The information entered by an end user needs to be processed on server side.

Based upon the outcome, the server generates a response.

The data sent by client towards server is available in the object of type *HttpServletRequest*.

This data is sent in the form of **name=value** pair and it is commonly known as request parameter.

To fetch these parameters, *HttpServletRequest* interface provides a method: *String [value] **getParameter**(String [name])*

HTTP GET Request:

In case of HTML form processing, if the data is sent towards server using HTTP GET request, it gets appended to the URL.

This leads to security implications especially while sending sensitive information.

This problem can be addressed by making HTTP POST request because, in this case, data is not appended to the URL; rather it is sent along with the page body.

- | HTTP GET | Vs | HTTP POST |
|---|----|-----------|
| 1. In GET request, parameters are appended to URL whereas in POST, they are sent along with page body. | | |
| 2. In case of GET request, there is a limitation on data transfer (8 KB) whereas in case of POST, there is no limitation. | | |
| 3. In case of GET request, there is a limitation on URL length (255 chars) whereas in case of POST, there is no limitation. | | |

Collaboration:

When one component of a web application interacts with another, then it is known as Collaboration.

It brings 2 benefits:

1. Modularity
2. Reusability

In order to implement collaboration, *jakarta.servlet* package provides an interface: *RequestDispatcher*.

It provides 2 methods:

1. *forward()*
2. *include()*

In case of *forward()*, the control is shifted from CALLING resource to the CALLED resource and the CALLED resource sends the final response back to the client.

In case of *include()*, the control is shifted from CALLING resource to the CALLED resource, the response from CALLED resource is taken and then the control is shifted back to the CALLING resource and further the CALLING resource sends the final response back to the client.

In order to obtain a reference of type *RequestDispatcher*, *ServletRequest* interface provides a method: *getRequestDispatcher(String calledResourceURL)*.

Session (State) Management:

HTTP is a stateless protocol. It does not maintain conversational state with the client.

In a web application, to perform some transaction, client has to make multiple requests.

Across these requests, server needs to maintain the state with the client.

The process of maintaining the state with every client is known as Session Management or State Management.

Session:

A session is a logical connection established between a client and server.

For every new user, there is a new session.

Typically, the session is started for every user when the user logs-in to the system and it is ended when it logs out.

Session Management Techniques:

In general, there are 4 techniques available to manage a session:

1. URL Encoding
2. Hidden Form Fields
3. Cookies
4. Servlet API – *HttpSession*

Although there are several techniques available for managing session, the most frequently used is *HttpSession*.

HttpSession:

It is an interface from *jakarta.servlet.http*.

An object of type *HttpSession* is responsible for maintaining client specific information (state).

It provides relevant methods to handle the information e.g. *setAttribute()* and *getAttribute()*.

In order to obtain *HttpSession*, *HttpServletRequest* interface provides *getSession()* method.

This method is overloaded:

1. *HttpSession getSession()*
2. *HttpSession getSession(boolean)*

The 1st method returns an existing session if available; otherwise creates a new one and returns the same.

The 2nd method with *true* value behaves exactly same as that of the 1st.

The *false* value returns the session only if it is available.

If it is not available, it returns *null*.

How *HttpSession* works?

HttpSession internally makes use of a text-based resource known as Cookie.

Cookie is a simple text-based data created on server side; but gets stored on client side.

When a client makes a request, the *getSession()* method checks whether there is any cookie coming along with the request or not.

Since, there is no cookie available in the 1st request, *getSession()* understands that the client does not have any session assigned and hence it creates a new session.

Once the session is created, web container assigns a unique ID to that session which is known as Session ID.

Further, the container stores the Session ID and the actual session object in some *Map* based collection.

Then the container generates a cookie and stores the session ID into that cookie and further dispatches that cookie back to client via HTTP Response.

When the client makes sub-sequent requests, along with every request, the same cookie is sent back to server and server fetches session ID from that cookie and hence it understands that the client is an old client.

Working with *HttpSession*:

HttpSession interface provides several methods:

1. *isNew()*
2. *setAttribute()*
3. *getAttribute()*
4. *removeAttribute()*
5. *setMaxInactiveInterval(long timeInterval [seconds])*
6. *getId()*
7. *invalidate()*

Web Application Resources:

A web application consists of different types of resources:

1. *HttpServletRequest*
2. *HttpServletResponse*
3. *HttpSession*
4. *ServletConfig*
5. *ServletContext*

HttpServletRequest:

It is used to handle request specific information.

It is lost as soon as response is generated.

HttpServletResponse:

It is used to handle response specific information.

It is lost as soon as response is generated.

HttpSession:

It is used to handle client (user) specific information.

It is lost as soon as session is terminated.

ServletConfig:

It is used to handle servlet configuration specific information.

Its instance is created for every servlet.

ServletContext:

It is used to handle application specific information.

Its instance is created for every web application.

It is lost as soon as server is stopped or the application is undeployed.

JSP:

JSP stands for Jakarta Server Pages.

It is a web component used to extend the functionality of a web server.

It runs on server side and hence can be used to handle server side processing.

It can be used to generate dynamic web contents.

Why JSP?

Jakarta EE specification provides one more web component JSP that works exactly same as that of servlet; however, from implementation perspective it is completely different.

JSP looks more like HTML rather than Java. Hence web page designers can design elegant web pages using features of Java without writing Java code.

When the degree of presentation logic is higher than processing logic, JSP is used instead of servlet.

Even though JSP looks like HTML, it is completely different from HTML.

- | HTML | Vs | JSP |
|---|----|-----|
| 1. All HTML files have <i>.html</i> extension whereas all JSP files have <i>.jsp</i> extension. | | |
| 2. HTML runs on client side (web browser) whereas JSP runs on server side (web server). | | |
| 3. HTML can generate static content only whereas JSP can generate dynamic content. | | |

Life Cycle of JSP:

Like servlet, the life cycle of JSP consists of 3 stages.

Similar to servlet, there are 3 life cycle methods:

1. *jspInit()*
2. *_jspService()*
3. *jspDestroy()*

When the JSP is requested very 1st time, it gets loaded by web container.

Once loading is finished, it is compiled and translated into servlet.

The Java class required for this servlet is generated by web container.

This class provides the implementation of life cycle methods.

Once the class is generated, web container instantiates it and invokes *jspInit()* method.

This method can be used to handle initialization if any.

After *jspInit()*, container invokes *_jspService()*.

The *_jspService()* method gets invoked every time when the request is made.

When the server is stopped or the application is undeployed, web container invokes *jspDestroy()*.

JSP File Structure:

A JSP file can contain following code:

1. HTML Tags
2. Any Valid Java Code
3. JSP Tags

JSP Tags:

JSP Tags are divided into 3 categories:

1. Directives
2. Scripting Elements
3. Standard Actions

Directives:

They are denoted using `<%@ ..... %>` delimiters.

They are of 3 types:

1. page
2. include
3. taglib

page:

It is used to configure JSP page using several attributes like:

1. language
2. session
3. import
4. isThreadSafe
5. errorPage

6. isErrorPage

language:

Used to specify scripting language of the page.

If omitted, it defaults to *java* and if used, it can contain only one possible value: *java*.

session:

Used to specify whether the page participates in a session or not.

If omitted, it defaults to *"true"*.

The *"true"* value indicates that the page participates in a session and hence the implicit object named as *"session"* is available.

import:

Allows to import *public* declarations from a package other than *java.lang*.

isThreadSafe:

Indicates whether the page is thread-safe or not.

If omitted, it defaults to *"true"*.

It is exactly opposite to the thread-safety concept seen in multithreading.

errorPage:

Used to divert the control to another JSP if the current JSP contains some Java code that fires an exception.

isErrorPage:

Used to configure the page as an error page.

If omitted, it defaults to *"false"* and therefore the implicit object *"exception"* is not available.

Day 4:

include:

It is used to include resources like HTML, JSP, Servlet or even a simple text file in the current JSP page.

It is done using *"file"* attribute.

taglib:

It is used in the context of custom tags i.e. user defined tags.

Scripting Elements:

These are associated with writing a Java code in JSP.

They are of 3 types:

1. Declaration
2. Scriptlet
3. Expression

Declaration:

It is used to declare variables or define methods.

It is denoted using `<%! ..... %>` delimiters.

Scriptlet:

It is used to write any valid Java code.

It is denoted using `<% ..... %>` delimiters.

The code written inside scriptlet gets processed directly into the `_jspService()` method defined by the container generated servlet implementation class for that JSP.

Therefore, variables defined within a scriptlet become local variables to that method.

It is not possible to define methods within a scriptlet.

Expression:

It is used to fetch value of a variable.

It is denoted using `<%=<Expression>%>` delimiters.

It is also possible to invoke methods provided their return type is other than *void*.

Implicit Objects:

JSP specification (Web Server) provides several objects by default to the page author and hence these objects are referred as Implicit Objects.

All these object names are predefined and case sensitive.

1. *request*
2. *response*
3. *out*
4. *session*
5. *config*
6. *application*
7. *page*
8. *pageContext*

9. *exception*

request:

It is of type *jakarta.servlet.http.HttpServletRequest*.

It is used to handle request specific information.

response:

It is of type *jakarta.servlet.http.HttpServletResponse*.

It is used to handle response specific operations.

out:

It is of type *jakarta.servlet.jsp.JspWriter*.

It is used to send the response back to the client.

session:

It is of type *jakarta.servlet.http.HttpSession*.

It is used to handle client (session) specific information.

config:

It is of type *jakarta.servlet.ServletConfig*.

It is used to handle configuration specific information of the servlet generated by web container for JSP.

application:

It is of type *jakarta.servlet.ServletContext*.

It is used to handle application specific information.

page:

It is of type *java.lang.Object*.

It refers to the object of the servlet class generated by a web container for the current JSP page.

pageContext:

It is of type *jakarta.servlet.jsp.PageContext*.

It is a single object through which other objects can be obtained.

It is especially used while working with custom tags i.e. user defined tags.

exception:

It is of type *java.lang.Throwable*.

It is used to handle exception specific information.

It is not available by default. It is made available by using [*isErrorPage = "true"*] setting for *page* directive.

Standard Actions:

There are several tags available in JSP meant for handling standard operations and hence they are referred as Standard Actions.

All JSP standard actions follow common pattern:

`<prefix:suffix>` where *prefix* is always *jsp* and *suffix* is the actual tag name.

`<jsp:useBean>`:

It is used to instantiate a Java Bean.

Any class with fields, constructors, getters, setters is referred as a Java Bean.

It provides several attributes:

1. id
2. class
3. type
4. scope

id:

Used to assign an ID to the bean so that the bean can be referred further using this ID.

class:

Used to provide fully qualified name of the class of which the bean is to be instantiated.

type:

Used to provide fully qualified name of the super class type or even an interface.

If omitted, it defaults to the same *class*.

scope:

Used to provide scope of the bean.

There are 4 possible scopes: *page*, *request*, *session*, *application*.

If omitted, it defaults to *page*.

`<jsp:setProperty>`:

It is used to set values for the properties of the bean.

There are several options available by which values can be set.

E.g. *property = "*" option* is used to set values for all the properties.

`<jsp:getProperty>`:

It is used to get values of the properties of the bean.

Setting properties of the bean:

In order to set values for bean properties, *setProperty* action is used.

These values can be set using several options.

property = "" :*

In this option, *"*"* indicates all properties.

In this case, web container identifies the names of request parameters then it identifies names of bean properties using Java's reflection semantics and tries to match them.

Once the match is found, it searches for relevant setter methods in the bean class with Java's naming convention.

Once the method is found, invokes it. Whatever type conversion required, is also done by web container.

property = "<property-name>":

This option is used to assign a value for an individual property.

In this case also, request parameter names and bean property names must match.

property = "<property-name>" param = "<param-name>":

This option is used if property names mismatch with request parameter names.

It provides a mapping between them.

property = "<property-name>" value = "<value>":

This option is used to assign a value directly.

The value can be either hard coded or evaluated through some expression.

In case of *getProperty* action, web container invokes getter method of the bean.

Other Actions:

There are several other standard actions provided by JSP specification:

1. *<jsp:forward>*
2. *<jsp:include>*
3. *<jsp:param>*

<jsp:forward>:

It is used to forward the request to the next resource like HTML, JSP or even a servlet.

It provides *page* attribute that specifies URL of the next resource.

<jsp:include>:

It is used to include the response of the target resource.

The source page generates final response back to the client.

It provides *page* attribute that specifies URL of the next resource.

This means resources can be included by 2 ways: *include* directive and *include* action.

However both work differently. The "*include*" directive includes the resource at translation time whereas the "*include*" action includes the resource at request time.

Since the "*include*" directive includes the resource at translation time, it may result into old state in the response.

However, it brings performance optimization as resource inclusion happens just once i.e. during the 1st request.

In case of *"include"* action, since resource inclusion happens for every request, it guarantees that always the updated state of the resource is rendered.

However, it leads to performance implications as resource inclusion happens for every request.

Therefore, in general, while including static resources like HTML, *"include"* directive is used whereas in case of dynamic resources like JSP, *"include"* action is used.

<jsp:param>:

It is used to send additional request parameters if any at the time of either *forward* or *include*.

Therefore, it must be used in conjunction with either *<jsp:forward>* or *<jsp:include>*.

Java Frameworks:

In the context of Java programming model, technologies are mainly divided into 2 categories:

1. Specification
2. Framework

Specification:

It is a set of rules that must be followed by some 3rd party software.

E.g. Jakarta EE is a specification and servers like web server and application server follow that specification.

Framework:

It is a partial implementation or solution designed to address common problems.

It provides a basic foundation and depending upon the requirement, further customizations can be done.

There are several frameworks based upon Java available:

1. Struts
2. JSF
3. JUnit
4. JMockit
5. EasyMock
6. Mockito

Even though there are several Java based frameworks available, 2 of them are very popular: Hibernate and Spring

Hibernate:

Hibernate is an open source Java based framework meant for handling persistence layer of an application.

The layer responsible for handling DB interactions is known as Persistence Layer.

Why Hibernate:

Prior to Hibernate, the persistence layer used to be implemented using JDBC.

Even though JDBC was capable of handling data persistency, there were several challenges:

1. A lot of coding (boiler-plate code) for data persistency
2. Tightly Coupled
3. SQL is mandatory
4. Extra coding is required for performance optimization
5. Extra coding is required for transaction management
6. Does not support object modeling, it requires further efforts

Although, all these challenges can be overcome using Hibernate, internally it uses JDBC for data persistency.

Therefore Hibernate is not a replacement to JDBC; rather is an abstraction (wrapper) on the top of JDBC.

Principles of ORM:

ORM stands for Object to Relational Mapping.

It is a set of principles that ensures that the data held in domain objects gets mapped to the relational DB tables.

ORM has several principles:

1. Automated Persistence
2. Works upon Metadata
3. Support for Object Modeling
4. Support for Performance Optimization
5. Support for Query Operations

Automated Persistence:

It makes possible to handle data persistency (CRUD) without writing SQL queries.

Works upon Metadata:

Since everything is hidden from developer, by some means basic information is to be provided e.g. mapping between class and table, field and column etc.

Information about DB connection e.g. driver class name, URL, UID, Password also needs to be provided.

This is done with the help of metadata.

Support for Object Modeling:

ORM provides support for object modeling e.g. composition (containment), inheritance etc.

Support for Performance Optimization:

ORM provides several techniques by which performance can be optimized e.g. efficient fetching, caching etc.

Support for Query Operations:

ORM supports a query language that works upon classes and their properties rather than tables and their columns.

Any framework that follows all these ORM principles is called as ORM framework e.g. Hibernate.

Apart from Hibernate, there are several other Java based frameworks available which follow ORM principles:

1. Toplink
2. Eclipselink
3. IBatis

Hibernate Core API:

The core API of Hibernate belongs to the main package: *org.hibernate*.

The API mainly consists of:

1. *Session*
2. *SessionFactory*
3. *Configuration*
4. *Transaction*
5. *Query*

Session:

It is an interface from *org.hibernate* package.

It is a light-weight object responsible for handling CRUD operations.

It is an abstraction (wrapper) on the top of JDBC Connection.

SessionFactory:

It is an interface from *org.hibernate* package.

It is a heavy-weight object responsible for obtaining *Session*.

It is recommended to have only 1 per application.

Configuration:

It is a class from *org.hibernate.cfg* package.

It is used to configure Hibernate based upon metadata.

There are 2 types of metadata:

1. Mapping Metadata
2. Configuration Metadata

It is also used to obtain *SessionFactory*.

Transaction:

It is an interface from *org.hibernate* package.

It is used to manage transactions.

It maintains atomicity.

It must be used while performing DML operations.

Query:

It is an interface from *org.hibernate.query* package.

It is used for handling query operations.

Day 5:**Getting Started:**

In order to get started with Hibernate, it is necessary to setup development environment.

Development environment setup:

1. JDK
2. Relational DB (MySQL)
3. Hibernate specific libraries
4. IDE (Eclipse)

Since Hibernate is a Java based framework, its libraries are in the form of several *.class* files.

These *.class* files are packaged into several *.jar* (Java Archive) files.

Therefore, it is necessary to add the relevant *.jar* files into project's build path.

Generally, every framework has its *.jar* files available on internet and they can be downloaded manually using the official website.

Instead of downloading them manually, it is preferred to get them using some build tool like Maven, Gradle etc.

Create an Entity Class:

In the context of Hibernate, any class of which an object is stored as a record in DB table is referred as an entity class.

The field that is being expressed as an identity, ideally has to be declared of class-type e.g. Integer instead of *int*.

Create a Mapping Metadata:

The entity class present on application side needs to be mapped to the relevant table present on DB side.

Its fields need to be mapped to the relevant columns of the table.

This is done using Mapping Metadata.

The mapping metadata can be provided by 2 ways:

1. Using XML File
2. Using Annotations

In case of XML, the file is referred as HBM file i.e. Hibernate Mapping.

This file can have any name but by convention it is *<Entity-class-name>.hbm.xml*.

E.g. if the entity class name is *Student* then the file name will be *Student.hbm.xml*.

This file must be placed under *src/main/resources* folder.

Create a Configuration Metadata:

Since Hibernate is responsible for establishing the connection with DB, it needs the relevant DB configuration setup e.g. driver class name, URL, username, password etc.

This is done by providing Configuration Metadata.

It can be done by 2 ways:

1. Declaratively
2. Programmatically

In case of declarative approach, there are 2 further options available:

1. Using *properties* file
2. Using XML file

In case of *properties* file option, the file name must be *hibernate.properties*.

In case of XML option, the file is referred as CFG file.

By default, Hibernate refers to a file with name *hibernate.cfg.xml*. If the file name is other than this, it is necessary to intimate Hibernate about the same.

This file must be placed under *src/main/resources* folder.

Create Main Class:

1. Configure Hibernate
2. Obtain *SessionFactory*
3. Obtain *Session*
4. Create an object of an Entity class
5. Obtain *Transaction* and start the same
6. Store the entity object into the session
7. Commit the *Transaction*
8. Close the *Session*
9. Close the *SessionFactory*

Hibernate Workflow:

When the object of *Configuration* class is created, Hibernate searches for DB configuration setup through a file: *hibernate.properties*.

If found, it loads entries from that file.

If entries are present inside XML file instead of *properties* file, it is necessary to intimate Hibernate about the same.

This is done by calling *configure()* method.

By default, this method prompts Hibernate to look for the XML file with name: *hibernate.cfg.xml*.

In order to load entries from the XML file with name other than *hibernate.cfg.xml*, overloaded *configure()* method is used.

Once Hibernate is configured using *Configuration* class, the same class object is used to obtain a *SessionFactory*.

This is done using *buildSessionFactory()* method.

Once *SessionFactory* is obtained, it is used further to obtain a *Session*. This is done using *openSession()* method.

Once *Session* is obtained, it is further used to perform CRUD operations. In case of DML operations, *Transaction* is must.

The *Transaction* is created and started using *beginTransaction()* method.

The *Session* interface provides relevant methods for handling CRUD operations e.g. in case of CREATE, *persist()* method is used.

Changes made to the *Session* need to be reflected towards DB so that the DML operation is considered to be successful.

This is done by committing the transaction using *commit()* method.

Once the operation is over, *Session* is closed and when the application is about to terminate, *SessionFactory* is closed.

This is done using *close()* method of both: *Session* and *SessionFactory*.

Metadata without XML:

Although XML brings flexibility, loose coupling in the architectural design, it brings several challenges:

1. It is verbose (lengthy)
2. It is time-taking
3. It is error-prone

To address these challenges, there are alternatives provided by Hibernate.

In case of mapping metadata, annotations can be used whereas in case of configuration metadata, programmatic approach can be used.

Mapping Metadata using Annotations:

It is possible to provide mapping between class and table, field and column using Annotations.

There are 2 categories of Annotations:

1. Hibernate Annotations
2. JPA Annotations

Hibernate Annotations:

These are the annotations provided by Hibernate itself.

JPA Annotations:

JPA stands for Jakarta Persistence API.

It is a specification and it is implemented by several ORM frameworks e.g. Hibernate, Toplink, Eclipselink, IBatis and so on.

JPA provides several annotations where following are frequently used:

1. *@Entity*
2. *@Table*

3. *@Id*
4. *@Column*

@Entity:

Applied at the class level to mark the class as an entity class.

It is mandatory.

@Table:

Applied at the class level of the entity class to map the same to the corresponding DB table.

It is optional. If omitted, it defaults to the unqualified name of the entity class.

@Id:

Applied either at the field level or getter method level of that field.

It is used to declare that field as an identity.

It is mandatory.

@Column:

Applied either at the field level or getter method level of that field.

It is used to provide the name of the DB column to which the field is to be mapped.

It is optional. If omitted, it defaults to the field name.

Programmatic Configuration:

Hibernate provides an alternative to declarative configuration.

Instead of making entries into *properties* or even XML file, it can be done by writing a Java code and therefore it is known as programmatic configuration.

This is done by using relevant methods of *Configuration* class.

Retrieving an Entity:

The entity that is persisted (stored) into the DB table can be retrieved at any time as per the requirement.

Typically, it is done against ID.

The *Session* interface provides *find()* method that is used to find an entity against its ID.

Since it is not a DML operation, *Transaction* is not required.

Updating an Entity:

In update operation, first the entity that is to be updated needs to be loaded.

Once the entity is loaded, its state can be modified by invoking setter methods.

However, it changes the state on Java side but not on DB side.

In order to complete the update operation, the modified state needs to be reflected on DB side.

This is done by committing the transaction.

Deleting an Entity:

In delete operation, first the entity that is to be deleted needs to be loaded.
The loaded entity can be deleted using *remove()* method of *Session* interface.
The removal is done upon committing the transaction.

Entity Persistence Life Cycle:

The life cycle of an entity goes through several stages which are associated with a single object that is called as Persistence Context.

A persistence context is a single object solely responsible for handling persistence related operations (CRUD).

In case of JDBC, it is *Connection* whereas in case of Hibernate, it is *Session*.

There are 4 stages involved in the life cycle:

1. TRANSIENT
2. PERSISTENT
3. DETACHED
4. REMOVED

TRANSIENT:

When the entity is not yet associated with the persistence context then it is said to be in TRANSIENT state.

At this moment, the entity does not have its identity.

PERSISTENT:

Once the entity is associated with the persistence context then it is said to be in PERSISTENT state.

At this moment, the entity has its own identity.

DETACHED:

If the session with which the entity is associated gets closed, that entity enters into DETACHED state.

REMOVED:

If the entity is permanently deleted then it enters into REMOVED state.

Working with *Session*:

In Hibernate, the most important object is *Session* as it handles all the persistence related operations (CRUD).

It provides relevant methods to achieve this e.g. *persist()*, *find()*, *remove()*.

Sometimes, there might be long running transactions and in such cases, if the *Session* is kept open, it causes performance implications.

Therefore it is recommended to close the session for performance optimization.

Once the *Session* is closed, the entity goes into DETACHED state.

Whatever changes made into that entity need to be reflected into the DB.

In order to handle this, the entity must be in PERSISTENT state.

In order to bring the DETACHED entity back into the PERSISTENT state, *Session* interface provides *merge()* method.

Day 6:

Associations:

In a business application, an entity might be associated with another entity or even several entities.

E.g. a single Department has many Employees, a single Teacher can teach multiple Subjects and a single Subject can be taught by many teachers.

When one entity is associated with another entity or several entities, that association needs to be mapped and therefore it is called as Association Mapping.

Associations are also known as Relationships.

In object oriented programming model, there are 4 types of relationships:

1. One to One
2. One to Many
3. Many to One
4. Many to Many

Every relationship is further divided into 2 types:

1. Unidirectional
2. Bidirectional

This means, there are total 8 relationships; but in reality they are 7 because One to Many Bidirectional and Many to One Bidirectional are same.

Unidirectional Relationships:

In this case, one entity holds a reference or references to another but the 2nd entity does not hold any reference to 1st.

One to One Unidirectional:

It is the simplest form of association.

In this case one entity holds a single reference to another but the 2nd entity does not hold any reference to 1st.

E.g. a relationship between Person and Passport.

In order to implement One-to-One association, JPA provides *@OneToOne* annotation.

It is to be applied either at the field level or getter method level of that field.

In order to map the foreign key column, JPA provides *@JoinColumn* annotation.

It is to be applied either at the field level or getter method level of that field.

One to Many Unidirectional:

In this case, one entity holds multiple references to another but the 2nd entity does not hold any reference to 1st.

E.g. relationship between `IplTeam` and `IplPlayer` where one IPL team has many IPL players and given team ID, players information is to be displayed but given player ID, team information is not required to be displayed.

In order to map one-to-many association, JPA provides *@OneToMany* annotation.

It is to be applied either at the field level or getter method level of that field.

The same annotation *@JoinColumn* is to be used for providing the name of foreign key column.

Bidirectional Relationships:

In this case, both the entities hold reference or references to each other.

It is necessary to set both ends of the association otherwise it leads to the problem of data inconsistency.

Since both ends of the association are set, Hibernate tries to perform updating the same foreign key column twice, from both the ends.

This increases the number of DB calls unnecessarily and leads to performance implications.

To address this problem, it is necessary to specify one side as the OWNING side and another side as the INVERSE side.

Many to Many Bidirectional:

In this relationship, both entities hold references to each other.

E.g. relationship between `Candidate` and `Certification`.

In case of Many-to-Many association, a junction table i.e. join table is required.

To map Many-to-Many association, JPA provides *@ManyToMany* annotation.

It is to be applied either at field level or getter method level of that field.

In order to configure junction table, JPA provides *@JoinTable* annotation.

It is to be applied either at field level or getter method level of that field.

This annotation is used in the class mapping of the entity class that acts as an OWNING side.

The *mappedBy* attribute of *@ManyToMany* annotation is used to indicate that the entity is an INVERSE side.

Performing Query Operations:

In a business application, retrieving data from DB table is a very common and frequent requirement.

The *Session* interface provides *find()* method to achieve this.

Even though it can be used to retrieve data from DB table, it has some limitation: It returns a single object because it finds it based upon ID.

In order to fetch multiple objects (records), Hibernate provides a query language known as HQL (Hibernate Query Language).

It internally works on the top of SQL.

It does not use table and its columns; rather it uses an entity class and its properties.

Just like SQL, it provides several clauses out of which 2 of them are frequently used: FROM and SELECT

FROM Clause:

It is the simplest form of HQL.

It directly returns objects of an entity class and therefore processing becomes very easy.

In order to execute HQL query, Hibernate provides *Query* interface from *org.hibernate.query* package.

Its reference is obtained using *createQuery()* method of *Session* interface.

Since the entire data is loaded, if it is not utilized, it leads to memory implications.

To address this problem, HQL provides SELECT clause.

SELECT Clause:

It is used to select specific fields of an entity.

Since memory is allocated as per the requirement it resolves the problem of memory implication.

However it is difficult to process as the query returns objects of an array type and the elements of array are of type *java.lang.Object*.

Working with Constructor Expression:

Sometimes, it is required to load same type of data very frequently.

In this case, HQL allows to populate an object of some Java Bean and use it further. This simplifies data processing.

It is done using a special feature known as Constructor Expression.

Day 7:

Spring:

Spring is a Java based framework meant for handling various aspects of the application.

Unlike Hibernate, Spring is not intended for doing only one thing; rather it is used for handling several things.

E.g. It provides support for integrating enterprise services within the application.

Why Spring:

Prior to Spring, enterprise services could be integrated with the application using Jakarta EE platform, especially through its component: EJB (Enterprise Java Beans).

Although it was possible to avail enterprise services via EJB, there were several challenges e.g. the coding was too much complicated.

This problem was resolved by Spring and therefore it got adopted rapidly.

Like Jakarta EE, Spring is completely based upon Component Driven Architecture.

In general, Java SE consists of unmanaged components whereas Jakarta EE consists of managed components.

In Java SE, component specific classes may be independent of any API i.e. they can be simple POJO (Plain Old Java Object).

In Jakarta EE, component specific classes always need to be dependent upon some API e.g. servlet specific class must inherit from *HttpServlet* and therefore it is not a POJO.

That means in Java SE, components are Unmanaged but they can be POJO whereas in Jakarta EE, components are Managed but they cannot be POJO.

Spring provides a dual service i.e. it can manage the component even though it is POJO.

Spring Modules:

Spring can handle various aspects of the application through several modules:

1. Spring Core
2. Spring AOP
3. Spring Data Access
4. Spring MVC
5. Spring REST
6. Spring Security
7. Spring Cloud
8. Spring AI

Spring Core:

It is the fundamental unit of Spring's ecosystem.

It is the module present at the base and all the other modules depend upon this module.

Spring AOP:

AOP stands for Aspect Oriented Programming.

It is a set of principles or it is a programming model which ensures that secondary concerns are decoupled (separated) from primary concerns.

It brings 2 benefits:

1. Modularity
2. Reusability

Spring Data Access:

Spring does not provide its own implementation for accessing data from DB.

It provides hooks (connectors / adapters) for various persistence related technologies like JDBC, Hibernate, and JPA and so on.

Spring MVC:

MVC stands for Model-View-Controller.

It is a commonly used design pattern for building web applications.

Spring MVC is used to building web applications using features of Spring.

Spring REST:

REST stands for Representational State Transfer.

It is a format or a pattern by which resources are represented on client side in any form that is suitable to the client.

Spring REST is used to building Restful Web Services or REST APIs.

Spring Security:

Spring Security is used to acquire security services within the application.

It is used to secure resources within the application.

Typically, it is implemented at 2 levels:

1. Authentication
2. Authorization

Spring Cloud:

It is used to acquire cloud based services.

Spring AI:

It is used to integrate AI based models or services.

Getting Started with Spring Modules:

Spring Core:

It is the module present at the base and it handles core functionalities of Spring:

1. Bean Management
2. Dependency Injection

Bean Management:

Spring can manage life cycle of the component even though it is simple POJO.

In order to achieve this, it is necessary to register an object of that component specific class with the environment of Spring.

Any object that is declared to Spring's environment or registered with Spring's environment is known as a Bean or Spring Bean.

Once the bean is registered, Spring can manage it and hence it is called as Bean Management.

Getting Started:

In order to get started with Spring, it is necessary to setup the development environment.

The setup mainly requires:

1. JDK
2. Spring Libraries
3. IDE (Eclipse)

Since Spring is a Java based framework, its libraries are available in the form of several *.class* files which are further distributed across several *.jar* files.

Therefore, to get started with Spring, it is necessary to bring the required *.jar* files into project's build path.

This can be done by downloading them manually using the official website or resolving them using some build tools like Maven, Gradle etc.

Create an Interface.

Spring promotes programming by interfaces.

Create an Implementation Class.

Register a bean of that class type with Spring's Environment:

Until the bean of the implementation class type is registered with Spring's environment, that bean is an Unmanaged Component and therefore, in order to get it managed by Spring, it needs to be registered.

There are 2 options available for bean registration:

1. Using XML
2. Using Annotations

Using XML:

In this case, the entry for bean registration is made using an XML file.

The file can have any name but by convention, it is *spring-config.xml*.

This file can be placed anywhere i.e. within the project or even outside the project.

Placing the file within the project is recommended.

It can be placed anywhere within the project; but ideally it is to be placed under *src/main/resources*.

Write a Main Class:

Once the bean is registered in the configuration unit of Spring, at any time it can be obtained back.

In order to achieve this, Spring provides a special service: *ApplicationContext*.

It is an interface and it is implemented by several classes.

The most commonly used implementation class is *FileSystemXmlApplicationContext*.

In order to obtain the bean, *ApplicationContext* interface provides *getBean(String ID)* method.

Working with *ApplicationContext*:

Spring provides a special service *ApplicationContext* for obtaining beans from configuration unit.

It is an interface implemented by several implementation classes like:

1. *FileSystemXmlApplicationContext*
2. *ClassPathXmlApplicationContext*

FileSystemXmlApplicationContext:

It is used to load beans from XML based configuration unit located on the file system.

It uses path of the configuration unit.

ClassPathXmlApplicationContext:

It is used to load beans from XML based configuration unit located in project's classpath.

In order to achieve this, the XML file must be present in project's classpath.

If the XML file is located under *src/main/resources*, it gets added into project's classpath automatically.

Dependency Injection:

The beans declared or registered in Spring's environment may have dependencies.

In this case, Spring has to inject values into the dependent properties of the bean and return that bean back to the client program.

This technique is known as Dependency Injection (DI).

Dependency Injection (DI) is also known as Inversion of Control (IOC) as the control is shifted from Developer to Spring's Container.

Spring can inject values into the bean properties using 2 options:

1. In injects values into the bean properties by invoking relevant setter methods.
Therefore, it is known as Setter Injection.
2. In injects values into the bean properties by invoking parameterized constructor.
Therefore, it is known as Constructor Injection.

Bean Loading:

The bean registered in the configuration unit is loaded by Spring's environment. In this case, Spring creates an object of the bean class and this process is known as Bean Loading.

Bean Loading is divided into 2 types:

1. EAGER
2. LAZY

EAGER:

This is the behavior Spring follows by default.

Spring creates an object of a bean class as soon as the *ApplicationContext* is initialized.

It does not wait for client to make a request for the bean.

LAZY:

In this case, Spring creates an object of a bean class only when client program requests for the bean.

Therefore, this type of loading is known as Lazy Loading.

It needs to be enabled using `lazy-init = "true"` setting at XML level.

Bean Scope:

The bean declared to Spring's environment has some scope.

There are 5 types of scope possible:

1. singleton
2. prototype
3. request
4. session
5. global-session

singleton:

It is the scope Spring follows by default.

If the same bean is requested multiple times, Spring returns same object.

prototype:

It is exactly opposite to singleton.

Spring creates a new object every time when client requests for the same bean.

request:

It is applicable only in Spring MVC environment.

Spring stores the bean into an object of type: *HttpServletRequest*.

session:

It is applicable only in Spring MVC environment.

Spring stores the bean into an object of type: *HttpSession*.

global-session:

It is applicable only in Spring Portlet environment.

Bean Configuration without XML:

Spring provides an alternative to XML for bean configuration because XML has several challenges:

1. It is verbose (lengthy)
2. It is time-taking
3. It is error-prone

To address these challenges, Spring provides support for Annotation based configuration.

There are 2 options available for bean configuration using annotations:

1. Java Based Configuration
2. Pure Annotation Based Configuration

Java Based Configuration:

In this approach, beans are registered in Spring's configuration unit using regular Java semantics i.e. using **new** keyword.

In order to provide configuration unit, it is necessary to define a Java class that is referred as configuration specific class.

This class can have any name but by convention it is *SpringConfig*.

Once the class is defined, it is necessary to intimate Spring that it is a configuration specific class.

This is done using *@Configuration* annotation.

It is to be applied at the class level.

Once the class is defined as a configuration specific class, it can be used to configure beans.

In order to configure a bean, it is necessary to define a method in such a way that it returns an object of the bean specific class.

Once the method is defined, it is necessary to intimate Spring that the method is a bean declaration method.

It is done using *@Bean* annotation.

It is to be applied at method level.

By default, Spring assigns an ID to this bean which is equal to the method name.

Loading Beans:

Once the beans are registered in Spring's configuration unit, any time they can be obtained back.

Spring provides a class: *AnnotationConfigApplicationContext* for loading the beans.

In order to register configuration specific class with *AnnotationConfigApplicationContext*, *register()* method is used.

After registration, to apply the changes, it is necessary to refresh the context which is done using *refresh()* method.

If the *Class* type of configuration specific class is passed as a parameter to *AnnotationConfigApplicationContext* constructor, there is no need to invoke *register()* and *refresh()* methods because it is handled internally.

Depending upon the requirement, it is possible to use several annotations supported by Spring e.g. *@Lazy*, *@Scope* etc.

@Lazy is used to enable lazy loading whereas *@Scope* is used to specify scope of the bean.

Both these annotations are to be applied at bean creation method level.

Day 8:**Bean Wiring:**

The bean declared to Spring's environment may depend upon another bean.

In this case, when the client program makes a request to Spring for getting the outer bean, Spring has to build inner bean, inject it into outer bean and then return that outer bean back to the client.

The process of injecting inner bean into outer bean is known as Bean Wiring.

Bean Wiring is of 2 types:

1. Explicit
2. Implicit

Explicit Wiring:

In this case, which bean is to be wired is mentioned explicitly.

Implicit Wiring:

In this case, instead of wiring the bean explicitly, Spring automatically figures it out and wires the bean.

Therefore this type of wiring is also known as auto wiring.

In order to implement auto wiring, Spring provides *@Autowired* annotation.

It can be applied either at the field level, setter method level of the field or even at constructor level.

How Auto Wiring works?

Spring identifies type of the field that is being auto wired using Java's reflection semantics.

Once the type is identified, it checks for the bean of that type in the configuration unit.

Once found, wires that bean.

What if the bean is not found:

If the bean is not found, *UnsatisfiedDependencyException* is thrown because the default behavior of *@Autowired* is MANDATORY.

To make it OPTIONAL, use *[required=false]* setting.

What if multiple beans are found:

If multiple beans of same type are found, it results into an ambiguity and generates the same exception.

In order to resolve this ambiguity, *@Primary* annotation is used.

It is to be applied at the bean creation method level to mark that bean as Primary.

There is another annotation available for resolving the ambiguity: *@Qualifier*.

It is to be applied at the same level where *@Autowired* is used.

Pure Annotation Based Configuration:

This is another way of configuring beans in the configuration unit.

In case of Java based configuration, beans are declared in the configuration unit using regular Java semantics i.e. **new** keyword.

In pure annotation based approach, beans are declared in the configuration unit using reflection semantics.

Spring provides *@Component* annotation to declare any class as a Component.

Once any class is declared as a Component, its object gets managed by Spring.

Once a class is declared as a Component, Spring can pickup that class only when the package in which that class is present, gets scanned.

This is done using *@ComponentScan* annotation.

It is used to scan the package that contains classes annotated with *@Component* annotation.

It is to be applied at the configuration specific class level.

If the package name is not mentioned, by default, the same package in which configuration specific class is available gets scanned.

If the package name is mentioned explicitly, the default behavior is vanished.

In this case, it is necessary to specify the package name explicitly.

Java based configuration and Pure annotation based configuration:

Both of these options are used to declare beans to Spring's environment.

In case of legacy classes 1st option is used.

If the classes are defined with full intention that they are going to act as spring components then 2nd option is used.

Spring AOP:

AOP stands for Aspect Oriented Programming.

It is a set of principles or it is a programming model which ensures that secondary concerns are decoupled (separated) from primary concerns.

It brings 2 benefits:

1. Modularity
2. Reusability

These secondary concerns cut across several parts of the application and therefore they are also known as Cross-Cutting concerns.

AOP Terminologies:

In the context of Spring AOP, there are 5 terminologies used:

1. Advice
2. Join Point
3. Point cut
4. Aspect

5. Proxy

Advice:

It defines WHAT and WHEN of an aspect.

Join Point:

It is the probable location where an aspect CAN BE plugged-in.

Point Cut:

It is the exact location WHERE an aspect IS TO BE plugged-in.

Aspect:

It is the combination of Advice and Point Cut.

It defines WHAT, WHEN and WHERE.

Proxy:

It is a special object responsible for intercepting client requests.

It is a mediator between the client program and the actual target object (bean).

It is also responsible for controlling the workflow.

Weaving:

In AOP, client requests are intercepted by some mediator known as proxy.

The process of proxy generation is known as Weaving.

In general, weaving is of 3 types:

1. Compile Time
2. Class Load Time
3. Runtime

Compile Time:

In this case, proxy is generated as soon as the target class is compiled.

Class Load Time:

In this case, proxy is generated as soon as the target class is loaded.

Runtime:

In this case, proxy is generated during execution.

In Spring AOP, weaving always happens at runtime.

Advice Types:

In Spring AOP, advice is of 5 types:

1. Before Advice
2. After Returning Advice
3. After Throwing Advice
4. After Advice

5. Around Advice

Before Advice:

It is used to handle functionality before the execution of target method.

After Returning Advice:

It is used to handle functionality after successful execution of target method.

After Throwing Advice:

It is used to handle functionality when the target method throws some exception.

After Advice:

It is used to handle functionality irrespective of whether the target method executes successfully or not.

Around Advice:

It is not any separate advice; rather it is a single advice that combines all the 4 above mentioned advices.

AOP Support in Spring:

Support for AOP in Spring is provided by a 3rd party library known as AspectJ.

It provides several annotations for AOP implementation:

1. *@Aspect*
2. *@Before*
3. *@AfterReturning*
4. *@AfterThrowing*
5. *@After*
6. *@Around*
7. *@Pointcut*

@Aspect:

It is to be applied at the class level to mark the class as an aspect.

@Before:

It is to be applied at the method level so that the method gets invoked before the execution of target method.

@AfterReturning:

It is to be applied at the method level so that the method gets invoked after successful execution of target method.

@AfterThrowing:

It is to be applied at the method level so that the method gets invoked when the target method throws some exception.

@After:

It is to be applied at the method level so that the method gets invoked irrespective of whether the target method executes successfully or not.

@Around:

It is to be applied at the method level for the configuration of around advice.

@Pointcut:

It is to be applied at the method level to mark that method as a Pointcut.

This method does not participate in execution; rather it is used for forward referencing.

Enabling Proxy Generation:

Spring generates a proxy for the target object; however by default proxy generation is disabled and until it is enabled, Spring cannot generate the proxy.

In order to enable proxy, Spring provides an annotation:

@EnableAspectJAutoProxy.

It is to be applied at the configuration specific class level.

Getting Started:

Install AspectJ Library:

In order to get started with Spring AOP, it is necessary to add AspectJ library into build path.

1. aspectjweaver
2. aspectjrt

Create a target class:

This is the class for which an aspect is to be plugged in.

It is a bean implementation class and it handles primary logic.

Create an aspect class:

This is the class responsible for handling secondary logic.

Since it is an aspect specific class, it configures all the 3 sections: WHAT, WHEN and WHERE.

WHAT is defined by the method, WHEN is specified using the relevant advice specific annotation and WHERE is specified using some point-cut expression.

The point-cut expression can be specified using several designators where the most commonly used designator is **execution**.

Create a configuration class:

This is the configuration unit of the application and in AOP, proxy is required therefore it is necessary to enable proxy generation.

It is done by applying *@EnableAspectJAutoProxy* annotation.

Write the client program (Main Class).

Working with Around Advice:

Around advice is a combination of Before, After Returning, After Throwing and After advices.

Since it combines all the advices, it contains a single *public* method that handles the entire workflow.

The other methods might be defined as *private*.

All these *private* methods are invoked from the single *public* method; however, there has to be a proper demarcation.

E.g. in between Before and After Returning oriented methods, a control must be sent towards target.

This is done using an interface: *ProceedingJoinPoint*

It provides a method: *proceed()*.

This *public* method is configured as an around advice using *@Around* annotation.

Introduction to Spring Boot:

Spring Boot is a Java based framework that extends the functionality of Spring.

It handles the functionalities of Spring with minimal efforts and therefore it is widely used in Spring based projects.

Benefits:

1. Simplified Dependency Management
2. Minimized Configuration
3. Simplified Execution

Simplified Dependency Management:

Spring Boot resolves dependencies very efficiently due to which developer can focus upon the core business functionality.

Minimized Configuration:

Spring Boot allows to build projects with very less configuration because it is mainly handled by the framework itself.

Simplified Execution:

Spring Boot simplifies execution process especially in a server centric architecture.

E.g. in case of web application, some web server is mandatory.

Spring Boot provides that web server and it is known as embedded web server. It provides Apache Tomcat by default.

Getting Started:

In order to get started with Spring Boot, it is necessary to create a Spring Boot project.

There are various options available for creating Spring Boot project:

1. Spring Boot Starter Project
2. Spring Boot CLI
3. Spring Initializer

Spring Boot Starter Project:

Many IDEs provide a relevant plug-in for creating Spring Boot starter project e.g. STS

Spring Boot CLI:

It is a command line interface meant for building Spring Boot projects.

Spring Initializer:

It is a web interface that allows to configure and build the project online and further download the same on local machine.

Spring Boot API:

Spring Boot API mainly consists of:

1. *SpringBootApplication*
2. *SpringApplication*

SpringBootApplication:

It is an annotation that is to be applied at the class level.

It is a combination of 3 annotations:

1. *@Configuration*
2. *@ComponentScan*
3. *@EnableAutoConfiguration*

Since it combines *@Configuration* and *@ComponentScan*, the annotated class can be used as a configuration unit and it can also be used for component scanning.

SpringApplication:

It is a class used to bootstrap the application.

It provides *run()* method that kick starts the embedded Tomcat server if available.

The method returns an object of type: *ConfigurableApplicationContext* and therefore it can be used to obtain beans from the configuration unit.

Day 9:

Spring MVC:

Spring framework provides a separate module known as Spring MVC that is meant for building web applications.

MVC Basics:

MVC stands for Model-View-Controller.

It is a commonly used architecture or design pattern for building web applications.

Model:

It is responsible for handling business logic of the application.

It is also responsible for holding application specific data.

It is defined either using simple Java class or an EJB.

View:

It is responsible for handling presentation logic of the application.

Typically, it is defined using JSPs.

Controller:

It is responsible for controlling navigation or workflow of the application.

It is used to bridge the gap between model and view.

Typically, it is defined using servlets.

Types of MVC:

1. MVC 1
2. MVC 2

MVC 1:

In this approach, all the 3 aspects: Model-View-Controller are handled using JSPs.

Since all the aspects are handled by JSP, it is called as page-centric architecture.

MVC 2:

In this approach, all the 3 aspects: Model-View-Controller are decoupled (separated) from each other.

Since the entire navigation and workflow is handled by Servlet, it is called as servlet-centric architecture.

Understanding Spring MVC:

Spring MVC is an extension to MVC pattern.

It implements MVC along with Spring features.

Benefits:

1. Since the application is based upon Spring, its features like bean management, dependency injection, AOP can be easily integrated.
2. It is based upon MVC 2 architecture and therefore it enforces to follow best practices.

Apart from MVC 2, Spring MVC follows one more design pattern: Front Controller.

It is a single servlet available in the application that handles the entire workflow. All incoming requests first hit this controller and then further delegation takes place.

Spring MVC Components:

In Spring MVC, there are 5 types of components:

1. Front Controller
2. Controller
3. Model
4. View
5. View Resolver

Front Controller:

It is a single servlet present in the application responsible for handling the entire navigation and workflow.

It is predefined by Spring MVC and it is known as: *DispatcherServlet*.

It needs to be configured in such a way that all incoming requests hit this servlet.

Controller:

It is responsible for handling the request.

It may invoke a Model for processing that request.

It returns name of the view back to Front Controller.

It is user defined and implemented using a simple Java class.

Model:

It is responsible for handling business logic of the application.

It is also responsible for holding application specific data.

It is user defined and implemented using a simple Java class.

View:

It is responsible for handling presentation logic of the application.

It is user defined and implemented using JSPs.

View Resolver:

Name of the view returned from Controller back to Front Controller needs to be resolved so that the actual view gets rendered on client side.

This is done using view resolver.

It is predefined by Spring MVC.

There are several view resolvers available but the most commonly used view resolver is: *InternalResourceViewResolver*.

Getting Started:

In order to get started with Spring MVC, it is necessary to setup development environment:

1. JDK
2. Spring MVC specific libraries
3. Web Server (TomCat)
4. IDE (Eclipse)

It is possible to download the required software; however these things can be simplified using Spring Boot.

Create a Spring Boot project for web application and add Tomcat Jasper dependency into it.

Change the default port (8080) of embedded TomCat:

In order to change the port number, make relevant entry in *application.properties* file located under *src/main/resources*.

Configure Front Controller (*DispatcherServlet*):

In Spring MVC based application, *DispatcherServlet* acts as a front controller. It is responsible for accepting all incoming requests. It is predefined but needs to be configured accordingly.

In case of Spring Boot, it is already configured with URL '/'.

Define a View:

It is responsible for presenting the data.

It is defined using JSP page.

Define a Controller:

The request made by client needs to be processed and it is done by a controller.

In Spring MVC, any Java class can act as a controller.

However, it is necessary to intimate Spring that it is a controller specific class.

This is done by applying *@Controller* annotation at the class level.

Once the controller class is defined, it can be used to configure application specific URLs.

It is done by defining a method which handles the request against the URL and returns the view name.

This method must be configured as a request handling method.

It is done by applying *@RequestMapping* annotation at that method level.

Configure a View Resolver:

The value returned from Controller to Front Controller is the view name and not the actual view.

In order to render the actual view back to client, it needs to be resolved using some Spring specific predefined view resolver.

The most commonly used is *InternalResourceViewResolver*.

It mainly works upon 2 properties: *prefix* and *suffix*.

They are configured using *application.properties* file.

HTML Form Processing:

Client sends data towards application using some HTML form.

When it arrives on server side, it needs to be processed as per the requirement. This is known as HTML form processing.

Unlike servlet, the control implementation class does not have *doGet()*, *doPost()* methods; rather the method names are user defined.

Therefore, by default the object of type *HttpServletRequest* is not available but it is required for obtaining request specific data.

It is injected using Spring's DI feature.

Handling HTTP Requests:

In order to handle HTTP Request, *@RequestMapping* annotation is used.

It is to be configured based upon type of the request.

There is an alternative (better approach) to *@RequestMapping* available.

Spring provides relevant annotations for every type of HTTP request e.g. for GET it is *@GetMapping*, for POST it is *@PostMapping* and so on.

Capturing request parameters without *HttpServletRequest*:

Spring provides an alternative to *HttpServletRequest* for capturing request parameters.

In this case, it is necessary to declare variables in the request handling method depending upon the number of parameters.

However, to retrieve the values, it is necessary to bind request parameters with local variables declared in method's signature.

This is done using *@RequestParam* annotation.

It is to be applied at the local variable level.

Rendering data in view:

Once the request is processed, it might be required to render application specific data in the resultant view.

E.g. the user who has logged-in, showing the username in view.

To achieve this, first it is necessary to store the data into model.

Spring provides an interface: *Model* which is to be used by injecting its reference into request handling method.

In order to store data into model, the *Model* interface provides several methods e.g. *addAttribute()*.

By default, the attribute gets added into REQUEST scope.

For SESSION scope, *@SessionAttributes* annotation is used.

It is to be applied at controller specific class level.

Spring REST:

Spring framework provides a separate module known as Spring REST that is meant for building Restful Web Services.

What is a web service?

Web Service is a software system designed to have machine-to-machine inter-operable communication over the network.

Machine-to-Machine:

The service provider and service consumer are hosted on different machines however they can interact with each other.

Inter-Operable:

The service provider can define the service without knowing about details of consumer.

Even though the technology and platform are different, still they can communicate with each other.

Communication over the network:

The provider and the consumer communicate with each other using some network protocol.

Types of web services:

There are 2 types of web services:

1. SOAP Based
2. Restful

SOAP Based:

SOAP stands for Simple Object Access Protocol.

It is a standard used by client and server for communication.

The data is transferred or exchanged in the form of XML and sent using HTTP protocol.

Hence SOAP is a combination of HTTP and XML.

Restful:

In the last several years, there is an alternative known as Restful Web Services available for handling interaction between cross-platform applications.

REST stands for Representational State Transfer.

It is standard used by client and server for interaction and it is based upon plain HTTP protocol.

It emphasizes upon data of the application and hence it is called as information-centric.

Representational:

The state of a resource can be represented in any form that is suitable to the client or consumer.

There are several forms possible but the most popular form is JSON i.e. Java Script Object Notation.

State:

The state of a resource can be sent from server to client as well as from client to server.

Transfer:

The state populated on one side needs to be made available on another side. This is done by transferring it.

Implementing REST:

Since REST is completely based upon HTTP protocol, it is not tied to any specific language or platform.

All popular technologies e.g. Java, Dot Net, Python, Node JS provide support for REST implementation.

In case of Java, there are several options available for implementing REST:

1. Jersey Framework
2. Spring MVC Framework using Spring Boot
3. Spring Boot only

Jersey Framework:

It is an open source Java based framework meant for building REST APIs.

Developers having Java background but not Spring may use this framework.

Spring MVC Framework using Spring Boot:

This approach is used especially when REST resources are to be combined with existing web application.

Spring Boot Only:

This approach is used when the focus is completely on REST APIs.

Day 10:

Getting Started:

Create a Spring Boot project with Web Dependency and change the embedded Tomcat port number.

Create a REST Endpoint:

In order to create REST endpoints, it is necessary to define a Java class.

Like Spring MVC, the endpoints are configured using relevant request handling annotations e.g. *@GetMapping*, *@PostMapping* and so on and the class can be annotated with *@Controller* annotation.

However, since Spring REST emphasizes upon data of the application and not the view name, it is necessary to intimate server environment that data is to be returned and not the view name.

This is done using *@ResponseBody* annotation.

It is to be applied at the return-type level.

Although, this configures the endpoint as a REST endpoint, in all such configurations, *@ResponseBody* needs to be used.

There is an alternative to *@ResponseBody* available.

It is especially used when the emphasize is completely upon implementing REST endpoints.

Spring provides *@RestController* annotation which is a combination of *@Controller* and *@ResponseBody*.

Conversion between Java Object and JSON:

When a server sends data towards client in the form of Java object, by the time it arrives on client side, it is converted into JSON format.

Similarly when a client sends data towards server in the form of JSON, by the time it arrives on server side, it is converted into Java object format.

This conversion is done by using 3rd party library known as Jackson.

Spring Boot by default uses Jackson API for type conversion.

Layered Architecture in REST:

The data that is sent by server back to client is obtained from some permanent persistent store e.g. Database.

Similarly, when client sends data towards server, it is to be stored in permanent persistent store like Database.

In this case, it is necessary to implement DB interaction specific logic.

This can be done within the controller specific class itself but ideally it has to be separated and handled using some another class.

This is done by implementing layered architecture.

It mainly consists of 3 layers:

1. Controller Layer
2. Service Layer
3. DAO Layer

Controller Layer:

It is responsible for handling requests coming from client and sending response to the client.

It is implemented using a simple Java class annotated with *@RestController* annotation.

Service Layer:

It is responsible for handling business logic of the application.

It is implemented using a simple Java class annotated with *@Service* annotation.

DAO Layer:

DAO stands for Data Access Object.

It is responsible for interaction with data storage for handling CRUD operations.

It can be implemented using a simple Java class annotated with *@Repository* annotation but in real projects, it is not required.

Working with Parameterized URLs:

In the context of REST APIs, URLs may have one or many parameters as per the requirement.

These parameters are configured using *{ }* notation.

The placeholder used in *{ }* needs to be bound with the variable declared in the signature of request handling method.

This is done using *@PathVariable* annotation.

It is to be applied at the local variable level of method signature.

Sending Data towards Server:

Sometimes client wants to send data towards server for further operations e.g adding a new resource, updating the resource and so on.

In this case, client sends relevant types of HTTP requests e.g. POST for creation, PUT for updation, DELETE for deletion and so on.

In case of POST and PUT requests, client sends data in the form of JSON and on server side it is accepted in the form of Java object.

Capturing Data from client:

When client makes a POST request with some JSON data, that data is attached to HTTP Request.

When the request arrives on server side, the data present in HTTP Request needs to be captured and stored into some Java object.

This is done by using *@RequestBody* annotation.

It is to be applied at local variable level of method signature.

Making a POST request:

In real-time projects or full stack applications, REST clients are implemented using client side technologies e.g. Angular, React, Vue and so on.

In development environment, in order to verify the working of REST API, developer uses some dummy or fake REST client for simulation.

There are several dummy or fake rest clients available e.g POSTMAN, Google's chrome extension: Talend API Tester.

Working with HTTP Response:

While working with REST, the APIs must be implemented keeping consumer in mind.

Ideally, server must generate appropriate response back to client.

E.g. if any resource is not available, server must send status: 404 i.e. Not Found.

This can be handled by throwing some exception usually a user defined exception.

In order to configure response status, *@ResponseStatus* annotation is used.

It is to be applied at the exception specific class level.

Interacting with DB:

In order to interact with DB, Spring framework provides a module: Spring Data Access.

It provides adapters for different types of persistence related technologies like JDBC, Hibernate, JPA etc.

Even though any of these technologies is used, developer has to write some code for data persistency.

Although requirements differ from domain to domain, the overall code for persistency remains same.

The only changing part is the entity type and may be ID type.

Once the type of entity and ID is mentioned, the entire code for data persistency is auto-generated.

This is done using a separate module: Spring Data JPA.

It provides an interface: *JpaRepository*.

It is a generic interface that works upon 2 things:

1. Entity Type
2. ID Type

It is to be customized by creating its sub-interface that specifies the actual entity type and the ID type.

Once it is mentioned, the entire implementation class is provided by Spring Data JPA module.

The DB specific configuration is done using *application.properties* file.

Final Day:

Sending response status without Exception:

It is possible to send appropriate response status even without throwing an exception.

Spring provides a class: *ResponseEntity* for handling this requirement.

It provides several *static* methods for sending an appropriate response e.g. *notFound()*, *ok()* etc.

Performing UPDATE:

A client may want to update the existing data as per the requirement.

There are 2 types of UPDATE operation:

1. Full Update
2. Partial Update

Full UPDATE involves modifying values of all the fields of the entity excluding ID field.

It is done by making HTTP PUT request.

Partial UPDATE involves modifying values of some of the fields excluding ID field.

It is done by making HTTP PATCH request.

Spring Security:

Spring Framework provides a module Spring Security that is meant for securing resources within the application.

Security is one of the enterprise services to be integrated within the application.

Many applications need resources to be secured e.g. Banking, Stock Exchange etc.

In general, security is implemented at 2 levels:

1. Authentication
2. Authorization

Authentication:

It is the 1st step of security.

It decides whether a user is allowed to enter into the application or not.

Typically it is done on the basis of username and password.

Authorization:

It is the 2nd step of security.

It decides what privileges user possesses for accessing specific resources.

Security Model:

Almost all types of applications work on the basis of same security model.

In general, the security model consists of 2 entities:

1. User
2. Role

User:

It is the person who is using the system.

An application may have one or many users.

The number of users is not fixed.

Role:

It is the logical entity specific to the domain present in the system.

An application has fixed number of roles.

Getting Started:

Create a Spring Boot project with Web and Security dependencies.

Create a Controller implementation class for configuring REST Endpoints.

By default Spring Security provides a single user with a username: **user** and a password that is randomly generated in the server logs.

Apart from this, Spring Security provides following thing by default:

1. Login Page
2. Logout Page
3. /logout URL

Configuring Spring Security:

The default credentials provided by Spring Security are just for development purpose only.

They are not recommended for production because application developer wants to configure security as per the domain requirements.

In order to configure Spring Security, configuration file can be used e.g. application.properties or application.yaml.

By default, Spring Security secures every URL i.e. it might be existing or even it might be non-existing.

The 1st option i.e. security configuration using PROPERTIES or YAML file has several drawbacks:

1. It can configure just a single user whereas an application may have several users.
2. It has full control over authentication but not authorization.

To address these problems, security configurations can be further customized e.g.

1. Which resources are not to be secured
2. Which resources need only authentication
3. Which resources need authentication as well as authorization
4. Who are the users and what role they play in the application.

Spring Security can be configured using any configuration specific class e.g. a class annotated with *@SpringBootApplication* annotation.

Configuring Users:

In real-time application, users are present in database and Spring Security needs to load users from DB.

Users might be present in memory as well but in this case they need to be configured.

In order to configure users, Spring Security provides an interface: *UserDetailsService*.

It is necessary to declare a bean of type *UserDetailsService*.

While configuring users, it is necessary to encrypt passwords otherwise it leads to security concerns.

Encryption can be done using several algorithms out of which the most popular one is BCrypt.

Configuring URLs with Roles:

Once the users of the application are configured with their roles, it is necessary to configure which resources are to be secured against what roles, which resources are not to be secured etc.

To handle this configuration, Spring Security provides an interface: *SecurityFilterChain*.

It is chain of filters executing in a specific order e.g.

UPAF (User Password Authentication Filter)

CSRF (Cross Site Request Forgery Filter)

The configuration is done by declaring a bean of type: *SecurityFilterChain*.

Introduction to JWT:

JWT stands JSON Web Token.

It is a standard used by client and server for exchanging the information, validating the client by server based upon the token and so on.

JWT mainly consists of 3 parts:

1. Algorithm
2. Claims (Information)
3. Signature

Algorithm:

It is the algorithm used to generate the token.

There are several types of algorithms available whereas the most commonly used is HMAC or even SHA.

Claims:

It is the actual information or payload that contains properties like subject i.e. user, issue date and time, expiry date and time etc.

Signature:

Once the token is issued, it is signed by some signing authority and upon sub-sequent requests, the token is validated against the signature.

JWT Workflow:

When client sends credentials in the 1st request, upon validating the credentials, server generates a token and sends the same back to the client.

When client makes sub-sequent requests, sends the same token along with HTTP Request Header: *Authorization*.

When the token arrives on server side, server validates the token and access is granted to client.

-----The End-----